# Semester 1 Final Presentation: What We Can Implement

Intelligent Water Network Leak Detection and Localization | IIIT Kottayam | Guide: Dr. Divya Sindhu Lekha

## 1. Summary

Our slides promise a hybrid hydraulic and data-driven pipeline for detection, topology-informed localization and Top-k ranking, with a Semester 1 scope of 1-2 networks, normal plus single-leak scenarios, varied leak location, size and start time, and an interpretable baseline. The code in the repository delivers part of this: one network (Net3), four leak scenarios, a pressure-threshold detector, a pressure plus flow-divergence localizer, a FastAPI backend and a Next.js dashboard. The professor's feedback that the work is basic matches the gap between the slides and the code. This report lists what can realistically be added before the presentation and in what order, and what should honestly be left to Semester 2.

## 2. Where we are (checked against the repository after the latest pull)

| Area | Status in code | Note |
|---|---|---|
| Simulation | Done, one network | Net3 only. 7 scenarios in simulation/scenarios.json: 4 leaks (nodes 3, 15, 115, 121), demand up and down at node 15, pump 335 outage. |
| Detection | Basic | Fixed 0.5 m pressure-drop threshold against a baseline. It also fires on demand changes and pump outages; the API reports 'cause not classified'. |
| Localization | Done, simple | Score = peak pressure drop + 50 x positive flow divergence. No graph distance is used anywhere. |
| Evaluation | Minimal | evaluate.py reports detection count, Top-1, Top-5 and non-leak anomaly flags. No F1, localization error or inference time. |
| Backend / dashboard | Done | FastAPI and Next.js dashboard with network map and time-series. |
| Noise, missing data, cross-network | Not done | Planned for Semester 2 on slide 8. |

## 3. Mismatches between the slides and the code (fix before presenting)

- **Slide 7 marks 'Deviation + network distance' as confirmed.** The code uses no network distance. Either add it (section 4, item 3) or mark it as under evaluation.
- **Slide 7 says PostgreSQL storage.** backend/database.py uses SQLite (sql_app.db). Change the slide or switch the database.
- **Slide 7 lists F1, false alarm, localization error and inference time.** Only part of this is computed today.
- **SEMESTER_1_DOCUMENTATION.md claims 100% Top-1.** That was measured on two scenarios. Do not present it as general accuracy; rerun on the larger set and quote those numbers.

## 4. What we can implement for Semester 1

Ordered by value for the presentation. Items 1-4 are the core and match the Semester 1 scope on slide 8; the rest strengthen it against the 'basic' feedback.

| # | Implementation | What it shows | Files |
|---|---|---|---|
| 1 | Scenario set that varies leak location, size and start time. Sweep a leak at every junction at one size, a size sweep (small to burst) at a few representative nodes, and several start times. | Detection and localization are tested broadly, not on 2-4 cases. Directly the Semester 1 scope on slide 8. | simulation/scenarios.json , generate_dataset.py |
| 2 | Second network (Net1 or Net2, from the WNTR examples; Net1-Net6 were in the earlier commit) with its own baseline. | '1-2 representative networks' on slide 8; first evidence of generalisation. | data/networks/, simulation/ |

| # | Implementation | What it shows | Files |
|---|---|---|---|
| 3 | Add network distance to localization: use shortest-path distance on the pipe graph (networkx is already a backend dependency) to smooth scores over neighbours, and report the error as hops or metres from the true node. | Makes the 'topology-aware' claim true and gives a localization-error metric. | ml_pipeline/localizer.py, evaluate.py |
| 4 | Full metrics: detection precision, recall and F1; false-alarm rate on no-leak runs; Top-1, Top-3, Top-5; mean localization error; inference time per scenario. Break results down by leak size and node type (dead-end, hub, high demand). | Quantitative results slide. Shows where the baseline works and fails. | ml_pipeline/evaluate.py |
| 5 | Normal vs leak vs other events. Add features that separate them: how widely the pressure drop spreads, whether the loss is concentrated at one node, and unaccounted flow (divergence minus metered demand). Classify an anomaly as leak, demand change or pump/source event. Needs a check that WNTR's demand and leak_demand outputs are usable for this. | Answers the professor's question on telling a leak apart from demand change and equipment change. Needs more event types in the simulator. | simulator.py, new ml_pipeline/features.py, classifier.py |
| 6 | More operating-condition scenarios: global demand +/-30%, reservoir head change, tank start level, pump speed, closed pipe. Show the pressure and flow effect of each versus baseline. | Shows how tank, pump and reservoir changes affect the network, as asked. | simulator.py, scenarios.json |
| 7 | Adaptive threshold: replace the fixed 0.5 m with a threshold learned from normal variation per network. | Needed for the second network; removes a hard-coded constant. | ml_pipeline/detector.py |
| 8 | Dashboard: scenario picker grouped by type, predicted cause, Top-k list with evidence (pressure drop, flow divergence, distance), and a results page. | Live demo that matches the slides. | backend/routers/analysis_api.py, frontend/ |

## 5. What to leave for Semester 2

- Sensor noise and missing readings (slide 8 places these in Semester 2). Only a small noise test is worth showing in Semester 1.
- Multiple simultaneous leaks, ML or graph models (autoencoder, One-Class SVM, GNN), and sparse sensor placement using centrality.
- Full cross-network training and testing across 3 or more networks.

## 6. Suggested order and effort

| Step | Work | Estimate |
|---|---|---|
| 1 | Items 1 and 4: bigger scenario set and full metrics on Net3. Gives real numbers first. | 2-3 days |
| 2 | Item 3: network distance in the localizer and the localization-error metric. | 1-2 days |
| 3 | Items 6 and 5: operating-condition scenarios, then leak vs demand vs pump classification. | 3-4 days |
| 4 | Items 2 and 7: second network and adaptive threshold. | 2 days |
| 5 | Item 8 and slide updates: dashboard changes, fix the slide mismatches, results slides. | 2 days |

Estimates assume the simulations converge. Large leaks combined with other events may fail to converge in WNTR; test those cases early. Estimates are rough and depend on the presentation date, which I do not know.

## 7. Suggested presentation structure (add to the existing 9 slides)

- **Scenario design:** table of scenario groups and the 'leak location, size, start time' sweep.
- **Detection results:** detection F1 and false-alarm rate on normal and non-leak runs.
- **Localization results:** Top-1, Top-3, Top-5 and localization error, by leak size and node type, with the network map.
- **Normal vs leak:** one figure showing pressure and flow signatures of a leak, a demand change and a pump outage.
- **Failure analysis:** which cases the baseline misses (small leaks, hub nodes, high demand) and why. This motivates Semester 2.

- **Live demo:** dashboard run on one leak and one non-leak scenario.
- **Semester 2 plan:** noise, missing data, multiple leaks, ML or graph methods, cross-network testing.

All results will come from simulation, not real sensors. State this on the results slides. No new results are reported in this document; the numbers have to be produced by running the evaluation on the extended scenario set.

# Implementation Plan: Classifying Leaks vs Other Network Events

## Context

The professor says the current work is basic. The repo (after pulling `591944c`) can simulate leaks, a demand change at one node, and one pump outage on Net3. The detector (`ml_pipeline/detector.py`) only checks whether pressure dropped more than 0.5 m. The API says this itself: "anomaly detected; cause not classified" (`backend/routers/analysis_api.py:62`). It cannot tell a leak from a demand change or a pump outage. It has never been tested on tank, reservoir or pump setting changes, or on any network other than Net3.
The relevant research question is: **when pressure and flow change in a network, is it a leak or normal operation changing, and if it is a leak, where?** The plan below answers that in four steps: simulate many conditions, show how each affects the network, classify the cause, and test generalisation.

## What each event does to the network (the physics the classifier uses)

| Event | Pressure pattern | Flow / mass balance | Distinguishing signature |
|---|---|---|---|
| Leak | Biggest drop at one node, decays with distance | Unmetered loss at that node; source outflow up; leak flow follows sqrt(pressure) | Local, persistent, present at night, unaccounted flow |
| Demand change (local) | Local drop, similar shape to a leak | Extra flow is metered demand | Divergence is explained by metered demand, follows the diurnal pattern |
| Demand change (global, +/-30%) | Broad shift, whole network | All demands scale | Wide spatial spread, fully metered |
| Pump outage or speed change | Broad pressure drop on the pump's service area | Pump flow falls to 0, tanks start to supply | Wide spread, no node with unaccounted loss |
| Reservoir head change | Network-wide shift up or down | Source flow changes | Uniform shift, no local peak |
| Tank level or isolation | Shift around the tank zone, flow reversal on tank pipe | Tank net inflow changes sign | Tank flow signature |
| Valve closed / blocked pipe | Drop downstream, rise upstream, flow reroutes | No mass loss | Opposite-sign pressure change across the pipe |
| Pipe roughness increase (ageing) | Gradual broad drop, high flow zones first | No mass loss | Slow trend, no local sink |

Key point to verify while building: leak flow and metered demand are both extra outflow at a node. Link-flow divergence alone cannot separate them. The separating feature is **unaccounted flow = divergence minus metered demand**. WNTR exposes demand and leak demand separately in `results.node` (`demand`, `leak_demand`); confirm this in the first step. Without it, fall back to the temporal and spatial features.

## Phase 1: More event types in the simulator (about 2 days)

Extend `Simulator.run` in `simulation/simulator.py`, which already handles `leak`, `demand_change` and `pump_outage`. Keep the JSON scenario format in `simulation/scenarios.json` and the `elif event_type` structure.

- `global_demand`: scale demand for all junctions.
- `reservoir_head`: change `reservoir.head_timeseries` from a start hour.
- `tank_level`: set `tank.init_level`; `tank_isolate`: close the tank pipe with a control.
- `pump_speed`: set `pump.speed_timeseries`; keep `pump_outage`.
- `valve_close` / `pipe_block`: close or throttle a pipe (minor loss) with a time control.
- `roughness`: raise pipe roughness (ageing).
- `hydrant`: temporary large demand at a node.
- `combined`: a list of events in one scenario (for example leak plus pump outage), so classification is tested with confounders.
- Also store metered demand and tank levels from the results for each scenario, not just pressures and flows.

Reuse: `Simulator.extract_pressures/extract_flows`; add `extract_demands`, `extract_tank_levels`.

## Phase 2: Scenario matrix and multiple networks (about 2 days)

- Rewrite `simulation/scenarios.json` as a generated matrix, built by a small script `simulation/build_scenarios.py` instead of typing 100+ entries.
- Per network, generate: no-event baseline, leaks (dead-end, hub, high demand, low/high elevation, several sizes, several start times), local and global demand changes, pump outage, pump speed, reservoir head, tank, valve, roughness, hydrant, and combined events.
- Networks: Net3 (current), plus Net1, Net2 and Net6, which ship in the WNTR package examples (they were deleted from `data/networks/` upstream, but exist in git history `d9c2911`). Add a `network` field to each scenario and one baseline per network.
- Check node, pump, tank and reservoir IDs per network against the `.inp` file when building scenarios (the README already warns about this).
- Store results in `data/raw_simulations/<network>/` and write ground truth (event type, location, size, time) into the per-network manifest.
- Keep `generate_dataset.py`'s skip-if-exists behaviour and add `--network` and `--only` options.

## Phase 3: Classification of the cause (about 3 days)

New file `ml_pipeline/classifier.py`. Classes: `normal`, `leak`, `demand_change`, `pump_event`, `tank_event`, `reservoir_event`, `valve_block`, `ageing`.

- **Rule-based first** (interpretable, easy to defend to the professor): compute features from the baseline-vs-scenario residuals and use thresholds:
- Spatial spread: fraction of junctions with pressure drop above the threshold.
- Peak concentration: ratio of the largest node drop to the median drop.
- Unaccounted flow (divergence minus metered demand) at the peak node.
- Source and tank flow change; pump flow; reservoir head shift.
- Pressure sign pattern across a pipe (valve block).
- Night-time persistence (hours 0-5 change) and trend (slow vs step).
- **Then a learned classifier** (scikit-learn, already a dependency of the project): random forest on the same features, so the rules and the model can be compared. Reuse the residual calculation already in `analysis_api.py` and `evaluate.py` instead of rewriting it; move it into a shared `ml_pipeline/features.py`.
- **Localisation stays in** `localizer.py` but runs only when the classifier says `leak`. Change the score to use unaccounted flow instead of raw divergence so demand changes do not rank as leaks.
- Replace the fixed 0.5 m threshold in `detector.py` with a threshold learned per network from normal variation (baseline noise), so it works on other networks.

## Phase 4: Evaluation that shows relevance (about 2 days)

Extend `ml_pipeline/evaluate.py`:

- Confusion matrix over event classes (is a demand change called a leak? is a pump outage called a leak?).
- False-alarm rate on `normal` scenarios, including noisy-sensor versions.
- Leak Top-1 and Top-5, split by leak size, node type and network.
- **Generalisation tests:** train on Net3, test on Net1/Net2/Net6; train on some leak nodes, test on unseen nodes.
- Sensitivity: smallest leak size detected; effect of sensor noise and sparse sensors.
- Baseline comparison: the current threshold detector versus the new classifier, so the improvement is measurable.

Save tables to `data/raw_simulations/evaluation_results.csv` (existing output) and add a per-class summary.

## Phase 5: Backend and dashboard (about 2 days)

- `backend/routers/analysis_api.py`: return `event_type`, class probabilities, the feature values behind the decision, and a plain explanation, replacing "cause not classified". Add a scenario-list endpoint and a network selector; read scenarios from the manifest instead of a fixed list.
- `backend/schemas.py`: add the new fields. `frontend/src/app/page.tsx` and `frontend/src/components/NetworkMap.tsx`: scenario picker grouped by network and event type, show the predicted cause, and show affected-area shading for pump, tank and reservoir events (not just a leak marker).
- Check `frontend/AGENTS.md` and `frontend/CLAUDE.md` before editing the frontend; they may restrict how it is changed.

## Phase 6: Report material (about 1 day)

For the professor: one chart per event type showing its pressure and flow effect versus baseline, the confusion matrix, and the Net3-to-other-network results. Update the Leak_Scenarios.pdf and README to match what is actually built.

## Order if time is short

Phase 1 (event types) -> Phase 2 on Net3 only -> Phase 3 rule-based -> Phase 4 confusion matrix. Then other networks, the learned classifier and the dashboard.

## Risks

- Large leaks combined with other events may not converge in WNTR; test the hardest combined cases first.
- Demand and leak flow may not be separable if WNTR output lacks the demand data; the first task is to check this.
- All results are simulated, not real sensor data; say so in the report (the README already does).
- Net6 is large (11,000 lines); simulation time may be long, so run it last.

## Verification

- `python simulation/generate_dataset.py` finishes for each network; manifest row count equals the scenario count.
- Spot-check physics: a pump outage shows a broad pressure drop and zero pump flow; a leak shows a local peak and unaccounted flow; a demand change shows divergence fully explained by metered demand.
- `python ml_pipeline/evaluate.py` prints the confusion matrix, false-alarm rate, Top-1/Top-5 by group, and the old-vs-new comparison.
- Run the backend, call `/api/analyze/<id>` for one scenario of each class, and confirm `event_type` is correct.
- Open the dashboard at http://localhost:3000 and confirm the scenario picker, the cause label and the map shading work for one leak, one pump outage and one tank event.

# Water Leak Detection: Scenario Catalogue

EPANET Net3 benchmark (92 junctions, 2 reservoirs, 3 tanks, 117 pipes, 2 pumps) | 24 h extended-period simulation | WNTR pressure-dependent demand

## 1. Purpose

Semester 1 tested only two leak cases (node 15 and node 115). This catalogue defines **41 distinct scenarios** (S00-S40) so the detector and localizer are tested on many different situations. Every scenario has the same end goal: **find the node where the leak occurred**. Scenarios differ in where the leak is, how big it is, when it starts, how many leaks exist, and what other disturbances (heavy flow, blocked pipes, pump trips, sensor faults) are present at the same time.

## 2. How each scenario is built

Each scenario is one entry in a scenario list that `simulation/generate_dataset.py` loops over. It writes `<name>_pressures.csv` and `<name>_flows.csv` plus a ground-truth label (the true leak node list). Leaks use `node.add_leak(wn, area, start_time, end_time)`. Blockages use a closed pipe or a large minor-loss factor. Pump trips use a pump status change. Heavy flow uses an extra demand at a junction. Demand scaling uses the global demand multiplier. Sensor noise and sparse sensors are applied to the CSVs after simulation, so no extra simulation is needed.

## 3. What counts as success

**Top-1**: the highest-ranked node is the true leak node. **Top-5**: the true node is among the five highest-ranked. For multi-leak scenarios every true node must appear in the Top-K. For no-leak scenarios (S00, S28, S30, S32, S33) the system must raise **no leak alarm**; these measure the false-alarm rate. Levels (Easy to Very hard) are my estimates from the physics, not measured results; they will need to be checked once the scenarios are simulated.

## 4. Coverage summary

| Group | Scenarios | What it varies |
|---|---|---|
| A. Control cases | S00 - S00 (1) | No-leak reference |
| B. Single leak - different places in the network | S01 - S14 (14) | Leak location and node type (dead-end, hub, pump, tank, elevation, demand) |
| C. Same location, different leak size | S15 - S18 (4) | Leak size at a fixed node |
| D. Different timing of the leak | S19 - S23 (5) | Start time, duration and growth |
| E. Multiple leaks at the same time | S24 - S27 (4) | Number and arrangement of leaks |
| F. Blocked / heavy flow | S28 - S34 (7) | Blockages, heavy flow and pump faults (with and without a leak) |
| G. Operating-condition and data-quality variations | S35 - S40 (6) | Tank level, seasonal demand, sensor noise, sparse and faulty sensors |

# Scenario Catalogue

Leak area is the orifice area in square metres. 'Timing: 12:00 on' means the leak starts at hour 12 and stays until the end of the 24 h run. Node numbers are Net3 junction IDs.

## A. Control cases (no leak)

| ID | Scenario | Leak node(s) | Area (m2) | Timing | Setup (how to simulate) | Why it is unique / challenge | Level |
|---|---|---|---|---|---|---|---|
| S00 | Normal operation | None | - | 24 h | Baseline run, no events. | False-alarm check. System must report NO leak and no location. | Easy |

## B. Single leak - different places in the network (ground truth = the leak node)

| ID | Scenario | Leak node(s) | Area (m2) | Timing | Setup (how to simulate) | Why it is unique / challenge | Level |
|---|---|---|---|---|---|---|---|
| S01 | Dead-end, low demand | 15 | 0.05 | 12:00 on | Existing scenario. Outer branch, demand ~1 gpm. | Leak is the only thing moving flow at this node; clear signal. | Easy |
| S02 | Central 4-way hub | 115 | 0.03 | 12:00 on | Existing scenario. Degree-4 junction, demand ~52 gpm. | Pressure drop spreads to 4 neighbours, so localisation is ambiguous. | Medium |
| S03 | Dead-end, high demand | 253 | 0.02 | 12:00 on | Dead-end at elev 11 m, demand ~55 gpm. | Leak hidden inside a large legitimate demand. | Medium |
| S04 | Dead-end, mid demand | 131 | 0.02 | 12:00 on | Dead-end, demand ~43 gpm. | Different branch from S01; tests generalisation to another zone. | Medium |
| S05 | Busiest 4-way hub | 119 | 0.03 | 12:00 on | Degree-4, demand ~176 gpm (3rd highest). | Heavy flow through node masks the leak. | Hard |
| S06 | Zero-demand hub | 120 | 0.02 | 12:00 on | Degree-4 junction with zero demand. | Any flow loss is purely the leak; no demand noise at node. | Medium |
| S07 | Large-pipe node | 189 | 0.03 | 12:00 on | Degree-4 on the 0.76 m diameter trunk main (pipe 189). | Big pipes absorb leaks with small pressure change. | Hard |
| S08 | Pump outlet | 61 | 0.02 | 12:00 on | Node at outlet of pump 335. | Pump curve responds to the leak and distorts the pressure pattern. | Hard |
| S09 | Near Lake pump, high elevation | 10 | 0.01 | 12:00 on | Elev 44.8 m, low pressure (~16 m). | Low pressure gives low leak flow (Torricelli). | Hard |
| S10 | Low elevation, high pressure | 169 | 0.02 | 12:00 on | Elev -1.5 m, degree-4, zero demand. | High pressure drives large leak flow; strong signal. | Easy |
| S11 | Highest elevation consumer | 153 | 0.02 | 12:00 on | Elev 20.2 m, demand ~44 gpm. | Weakest pressure head among consumers. | Hard |
| S12 | Mid-network chain node | 127 | 0.02 | 12:00 on | Degree-3 inline node, elev 17.1 m. | Neighbours have similar response; Top-1 vs Top-5 gap is small. | Medium |
| S13 | Next to Tank 1 main | 40 | 0.02 | 12:00 on | Elev 40.2 m, joins 2.5 m dia. tank pipe. | Tank buffers pressure, so the signal is damped. | Hard |
| S14 | Remote low dead-end | 167 | 0.02 | 12:00 on | Elev -1.5 m, demand ~15 gpm, far end. | Far from pumps and tanks; flow signal travels a long way. | Medium |

## C. Same location, different leak size (node 109, highest demand, 231 gpm)

| ID | Scenario | Leak node(s) | Area (m2) | Timing | Setup (how to simulate) | Why it is unique / challenge | Level |
|---|---|---|---|---|---|---|---|
| S15 | Tiny seep | 109 | 0.0005 | 12:00 on | Incipient leak, near sensor resolution. | Below the 0.5 m detection threshold; tests sensitivity. | Very hard |
| S16 | Small leak | 109 | 0.002 | 12:00 on | Small orifice. | Marginal detectability against demand noise. | Hard |
| S17 | Medium leak | 109 | 0.01 | 12:00 on | Typical service-line crack. | Reference size for comparing S15-S18. | Medium |
| S18 | Pipe burst | 109 | 0.05 | 12:00 on | Large orifice. | Huge pressure drop network-wide; the whole region looks affected. | Easy to detect, hard to pinpoint |

## D. Different timing of the leak

| ID | Scenario | Leak node(s) | Area (m2) | Timing | Setup (how to simulate) | Why it is unique / challenge | Level |
|---|---|---|---|---|---|---|---|
| S19 | Night-time leak | 193 | 0.01 | 02:00 on | Start at 2 am, low demand, high pressure. | Minimum-night-flow condition; easiest time to see a leak. | Easy |
| S20 | Morning-peak leak | 193 | 0.01 | 08:00 on | Start during morning demand peak. | Leak coincides with a natural pressure dip. | Hard |
| S21 | Short burst | 193 | 0.03 | 14:00-15:30 | Leak with end_time set, then repaired. | Event is over before the day ends; detector must catch a transient. | Medium |
| S22 | Growing leak | 193 | 0.002 -> 0.01 -> 0.03 | 06:00, 12:00, 18:00 | Three stages at the same node, area grows with time. | Slowly worsening pipe crack; no sudden jump. | Hard |
| S23 | Very late leak | 193 | 0.02 | 23:00 on | Only 1-2 hourly samples after start. | Almost no post-leak data to localise from. | Hard |

## E. Multiple leaks at the same time

| ID | Scenario | Leak node(s) | Area (m2) | Timing | Setup (how to simulate) | Why it is unique / challenge | Level |
|---|---|---|---|---|---|---|---|
| S24 | Two far-apart leaks | 131 and 225 | 0.02 each | 12:00 on | Two dead-ends in different zones. | Two separate signatures; Top-K must contain both. | Medium |
| S25 | Two adjacent leaks | 171 and 173 | 0.02 each | 12:00 on | Two ends of the 0.76 m trunk pipe 189. | Overlapping signatures can merge into one. | Hard |
| S26 | Staggered triple leak | 109, 193, 243 | 0.01, 0.02, 0.01 | 06:00, 12:00, 18:00 | Three leaks start at different times. | Tests incremental detection of new leaks. | Hard |
| S27 | Big leak masks small leak | 115 and 253 | 0.05 and 0.003 | 12:00 on | One burst plus one tiny seep. | Burst dominates the score; small leak is hidden. | Very hard |

## F. Blocked / heavy flow (the water-flow problems in the project description)

| ID | Scenario | Leak node(s) | Area (m2) | Timing | Setup (how to simulate) | Why it is unique / challenge | Level |
|---|---|---|---|---|---|---|---|
| S28 | Heavy flow: hydrant flush | None (decoy at 199) | - | 10:00-12:00 | Add +1500 gpm demand at 199 (fire / flushing). | Large legitimate flow causes big pressure drop; must NOT be called a leak. | Medium |

| ID | Scenario | Leak node(s) | Area (m2) | Timing | Setup (how to simulate) | Why it is unique / challenge | Level |
|---|---|---|---|---|---|---|---|
| S29 | Leak + flush decoy | Leak 121; decoy 189 | 0.02 | 12:00 on; flush 10:00-14:00 | Real leak at 121 while 189 is flushed. | System must pick 121, not 189. | Very hard |
| S30 | Blocked pipe (closed valve) | None (blockage pipe 189) | - | 10:00 on | Close pipe 189 (171-173) with a control. | Downstream pressure falls like a leak; flow is rerouted. Not a leak. | Hard |
| S31 | Leak + blocked pipe | Leak 143; block pipe 321 | 0.02 | 12:00 on; block 08:00 on | Pipe 321 (163-265) closed, leak at 143. | Blockage changes flow paths, so baseline no longer matches. | Very hard |
| S32 | Partial blockage | None (pipe 330) | - | 08:00 on | Throttle pipe 330 (60-601) with a large minor-loss factor. | Gradual flow restriction and pressure loss; no mass loss. | Hard |
| S33 | Pump shutdown | None | - | 10:00-14:00 | Pump 335 set to closed. | Network-wide pressure drop; no node has mass imbalance. | Medium |
| S34 | Leak + pump shutdown | Leak 123 | 0.02 | 12:00 on; pump off 10:00-14:00 | Leak while pump 335 is off. | Pressure falls everywhere; only mass imbalance points to the leak. | Very hard |

## G. Operating-condition and data-quality variations

| ID | Scenario | Leak node(s) | Area (m2) | Timing | Setup (how to simulate) | Why it is unique / challenge | Level |
|---|---|---|---|---|---|---|---|
| S35 | Low tank levels | 151 | 0.02 | 12:00 on | Start Tanks 1-3 at 20% level. | Different baseline hydraulics; baseline must be re-simulated. | Hard |
| S36 | Heat-wave demand +30% | 151 | 0.02 | 12:00 on | Global demand multiplier 1.3. | Higher flows and lower pressures everywhere. | Hard |
| S37 | Winter demand -30% | 151 | 0.02 | 12:00 on | Global demand multiplier 0.7. | Higher pressures; checks the same detector works. | Medium |
| S38 | Noisy sensors | 15 | 0.05 | 12:00 on | Gaussian noise: 0.3 m on pressure, 2% on flow. | Realistic measurement error on the S01 case. | Hard |
| S39 | Sparse sensors | 131 | 0.02 | 12:00 on | Only 20 of 92 junctions monitored. | Real utilities do not sensor every node. | Very hard |
| S40 | Sensor dropout | 253 | 0.02 | 12:00 on | Leak-adjacent sensors report a stuck value. | Faulty sensor close to the leak. | Very hard |

# Reference: Net3 nodes used

| Node type | Net3 junctions | Why it matters |
|---|---|---|
| Dead-end (degree 1) | 15, 35, 131, 166, 167, 203, 219, 225, 231, 243, 253 | Leak affects one branch only |
| Hub (degree 4) | 111, 115, 119, 120, 121, 169, 189, 193, 255 | Leak signature spreads to four neighbours |
| Highest demand | 109 (231 gpm), 101, 119, 151, 111, 105, 103, 199 | Leak is masked by consumption |
| High elevation / low pressure | 10, 40, 20, 50, 153, 127 | Weak leak flow |
| Low elevation / high pressure | 167, 169, 143, 171, 173, 121 | Strong leak flow |
| Pumps | Pump 10 (Lake to node 10), pump 335 (node 60 to 61) | Pump trips and pump-outlet leaks |
| Trunk pipes (0.76 m) | 189 (171-173), 321 (163-265), 329 (61-123), 330 (60-601), 333 (601-61) | Blockage and large-pipe cases |

Note: the current detector uses a fixed 0.5 m pressure-drop threshold and compares to a clean baseline. Scenarios S15, S16, S35-S37 and S39-S40 will probably expose its limits; that is intended, and is useful material for the Semester 2 roadmap.

# Implementation Roadmap

Total estimate: about 8 working days. Phases are in order; each ends with a check you can run before moving on.

## Phase 1: Scenario definition (about 1 day)

• Create `simulation/scenarios.py` holding all 41 scenarios as data. Each entry has: id, name, list of leaks (node, area, start, end), list of events, global demand multiplier, noise setting, sensor mask and ground truth.
• An event is one of: hydrant, close_pipe, throttle_pipe, pump_off, tank_level.
• Keep ground truth in the same file, for example truth = ['109'], or [] for no-leak cases (S00, S28, S30, S32, S33).
• Check: the file imports and a test confirms every node and pipe ID exists in Net3.inp.

## Phase 2: Extend the simulator (about 2 days)

• Add a single `run(scenario)` method to `simulation/simulator.py`. It builds a fresh network, applies everything, and runs WNTRSimulator.
• Single, multiple and staggered leaks (S01-S27): `node.add_leak(wn, area, start_time, end_time)`, once per leak.
• Growing leak (S22): three leaks at the same node with increasing areas and later start times.
• Hydrant flush (S28, S29): add a temporary extra demand at the node for the time window.
• Blocked pipe (S30, S31): close the pipe with a time control. Partial blockage (S32): raise the pipe minor loss.
• Pump trip (S33, S34): time control that closes pump 335.
• Low tanks (S35): set tank init_level. Demand +/-30% (S36, S37): scale every junction demand multiplier.
• Baselines: S35-S37 and the pump or valve events change normal operation, so each needs a matching baseline run. Decide per scenario which baseline the detector compares against.
• Convergence risk: run S18, S29 and S34 first. Large leaks combined with disturbances may fail to converge in WNTR; if so, reduce the leak area or event size and note it.
• Check: S01 and S02 reproduce the existing CSVs.

## Phase 3: Dataset generation (about 1 day)

• Rewrite `generate_dataset.py` to loop over scenarios.py. Per scenario write `<id>_pressures.csv`, `<id>_flows.csv` and `<id>_meta.json` (ground truth and parameters) to data/raw_simulations/.
• Apply sensor noise (S38), sparse sensors (S39) and sensor dropout (S40) after simulation as post-processing with a fixed random seed. They need no extra simulation.
• Add a command-line option such as `--only S29` to regenerate one scenario.
• Check: 41 scenario folders or file sets exist and the meta files load.

## Phase 4: Connect the backend and dashboard (about 1 day)

• Change `backend/routers/analysis_api.py` to discover scenarios from the meta files instead of a hardcoded list, and to load the matching baseline per scenario.
• Add a scenario-list endpoint returning id, name, category, level and ground truth.

• Update the dashboard scenario dropdown to group scenarios by category (A-G).
• Check: /api/analyze/<id> works for every scenario.

## Phase 5: Evaluation (about 1 day)

• Extend `ml_pipeline/evaluate.py` to run all scenarios and report per scenario and per category: Top-1, Top-5, and for multi-leak cases the fraction of true leaks found.
• For no-leak scenarios report the false-alarm rate (alarm raised when none should be).
• Save the results as a CSV and a one-page summary table.
• Check: results table has 41 rows.

## Phase 6: Fix what fails (about 2 days)

• Expected weak spots: tiny leaks (S15, S16) under the fixed 0.5 m threshold; decoys and blockages (S28-S34) because the detector only looks for pressure drops and flow divergence; multiple leaks (S25-S27) because it returns one ranked list; sensor problems (S38-S40).
• Possible fixes: per-node thresholds from normal variation, a blockage check (flow change with no mass loss), iterative leak removal for multiple leaks, and training a classifier on the scenario data.
• Re-run the evaluation after each change and keep the before and after table; this is the Semester 2 material.

## Order of work if time is short

• Do first: S00-S14, S19-S21 (only need add_leak). Then S15-S18, S24-S27. Then S28-S34 (events). Last: S35-S40 (operating conditions and data quality).